

Today 11:56 AM

https://github.com/SETAPESU26/36_dinoRun

I am working on a Pygame Dino Run project. Analyze the attached/current game.py code and identify the bug responsible for the high score not updating after a collision. Do not rewrite unrelated parts of the program. Explain exactly what is wrong and give the minimal code change required.

I checked the current `game.py` in your repository. The bug is exactly in the high-score comparison inside `Game.update()` .  GitHub

The bug

After a collision, the code calculates the current run's score:

 Python

```
current = self.score // 10
```

Then it currently has:

 Python

```
if self.high_score > current:
    self.high_score = current
    save_high_score(self.high_score)
```

This comparison is **backwards**.

You want to update the high score when the **current score is greater than the previous high score**.

For example:

- Previous high score = 50
- Current run = 70

The current condition checks:

 Python

```
50 > 70    # False
```

So nothing happens.

Minimal fix

Change **only this line**:

</> Python

```
if self.high_score > current:
```

to:

</> Python

```
if current > self.high_score:
```

So the corrected section is:

</> Python

```
if hit:
    self.state = "lose"

    current = self.score // 10

    if current > self.high_score:
        self.high_score = current
        save_high_score(self.high_score)

    return
```

That's the **only change required for Task 1**. The repository's README explicitly identifies the deliberate bug as the reversed comparison immediately after collision detection.  [GitHub](#)

Do not change `current = self.score // 10` , `save_high_score()` , `reset()` , or any other part of the program.

Now implement Task 2 from the README. Implement `dino_tint(on_ground)` only. Make the dino use its normal green color while on the ground and a clearly different color while airborne. Keep the existing game architecture and do not modify unrelated code. Explain the change briefly.

Task 2 is already wired into the existing architecture: `Dino.draw()` calls `dino_tint(self.on_ground)` once per frame, and falls back to `self.color` if the function

returns `None` . [🌐 GitHub](#)

Replace **only** the `dino_tint()` function:

`</>` Python

```
def dino_tint(on_ground):
    """Return an (r, g, b) colour override for the dino based on whether it's on the ground, or N
    if on_ground:
        return (83, 141, 78)      # normal green
    return (70, 120, 220)        # blue while airborne
```

What this does

- **On ground (True)** → (83, 141, 78) , the dino's existing normal green.
- **Airborne (False)** → (70, 120, 220) , a clearly different blue.
- No changes to `Dino.update()` , `Dino.draw()` , jumping, physics, or any other code.

The README specifically says Task 2 is intended to use the `on_ground` boolean to return an RGB override, with `Dino.draw()` already handling the result. [🌐 github.com +1](#)

So do not modify anything except the **pass** inside `dino_tint()` .

Implement Task 3 from the README. Add a simple visual feedback effect when the dino successfully passes an obstacle. Use the existing `on_obstacle_passed(obstacle,score)` hook. I want a small "+1" popup near the dino that appears when an obstacle is passed and then disappears after a short duration. Do not break the existing scoring or obstacle logic, and do not modify unrelated functionality.

Task 3 can be implemented with a **small popup state** driven by the existing `on_obstacle_passed(obstacle, score)` hook. The hook is already called exactly once when an obstacle passes the dino, so we don't need to touch obstacle/scoring logic. [🌐 GitHub](#)

Make these **three small changes** in `game.py` .

1. Add popup state near the constants

Add this below `HIGH_SCORE_FILE` :

`</>` Python

```
PASS_POPUP_TEXT = None
PASS_POPUP_POS = (0, 0)
PASS_POPUP_UNTIL = 0
```

2. Implement `on_obstacle_passed()`

Replace its `pass` with:

`</>` Python

```
def on_obstacle_passed(obstacle, score):
    """Show a short +1 popup when an obstacle is successfully passed."""
    global PASS_POPUP_TEXT, PASS_POPUP_POS, PASS_POPUP_UNTIL

    PASS_POPUP_TEXT = "+1"
    PASS_POPUP_POS = (obstacle.rect.right + 5, obstacle.rect.top - 20)
    PASS_POPUP_UNTIL = pygame.time.get_ticks() + 500
```

This makes the popup last **500 ms**.

3. Draw the popup in `Game.draw()`

Add this after the obstacle-drawing loop:

`</>` Python

```
if PASS_POPUP_TEXT and pygame.time.get_ticks() < PASS_POPUP_UNTIL:
    popup = self.font.render(PASS_POPUP_TEXT, True, (50, 160, 50))
    screen.blit(popup, PASS_POPUP_POS)
```

So that portion becomes:

`</>` Python

```
self.dino.draw(screen)
for obs in self.obstacles:
    obs.draw(screen)

if PASS_POPUP_TEXT and pygame.time.get_ticks() < PASS_POPUP_UNTIL:
    popup = self.font.render(PASS_POPUP_TEXT, True, (50, 160, 50))
    screen.blit(popup, PASS_POPUP_POS)

score_surf = self.font.render(
    f"Score: {self.score // 10}    Best: {self.high_score}",
    True,
    (50, 50, 50)
)
```

Why this doesn't affect the existing game logic

The existing flow remains untouched:

 Python

```
if not obs.passed and obs.rect.right < self.dino.rect.left:
    obs.passed = True
    on_obstacle_passed(obs, self.score // 10)
```

The hook simply records **where and until when** the `+1` should be displayed. The existing `obs.passed`, score calculation, obstacle movement, and collision detection are unchanged.

 GitHub

One important detail: I would not put the popup into `self.score` or modify `self.score` inside the hook. The popup is purely visual, so the existing scoring mechanism remains completely independent.

Implement Task 4 from the README. Make `max_jumps()` allow the dino to perform a double jump. Return the appropriate integer and do not modify the existing jump-count bookkeeping or unrelated code. Explain why the change enables the double jump.

Task 4 requires only changing the empty `max_jumps()` hook.

Replace:

 Python

```
def max_jumps():
    """Return how many jumps the dino gets before it must land again (2 for a double jump), or No
    pass
```

with:

 Python

```
def max_jumps():
    """Return how many jumps the dino gets before it must land again (2 for a double jump), or No
    return 2
```

That's the **only change needed**. The README explicitly specifies `2` as the intended value for the double-jump feature.  GitHub

Why this enables double jump

`Dino.jump()` already contains the necessary bookkeeping:

 Python

```
if self.jump_count < (max_jumps() or 1):  
    self.vel_y = JUMP_FORCE  
    self.on_ground = False  
    self.jump_count += 1
```

With `max_jumps()` returning 2 :

- First jump: `jump_count = 0 < 2` → allowed → becomes 1
- Second jump while airborne: `1 < 2` → allowed → becomes 2
- Third jump: `2 < 2` → rejected
- When the dino lands, the existing `update()` resets `jump_count = 0` .

So you **do not need to modify `Dino.jump()` or `Dino.update()`** . The existing jump-count bookkeeping already supports exactly what Task 4 requires.  [GitHub](#)